

情報科学演習 D

課題 4

担当教員 : 梶井 晃基
提出者 : 鶴見 俊介
所属/学年 : 基礎工学部 情報科学科 3 年 計算機科学コース
学籍番号 : 09B23057
電子メール : u299151f@ecs.osaka-u.ac.jp

提出日 : 2026 年 1 月 23 日
締切日 : 2025 年 1 月 30 日 23:59

1 システムの仕様

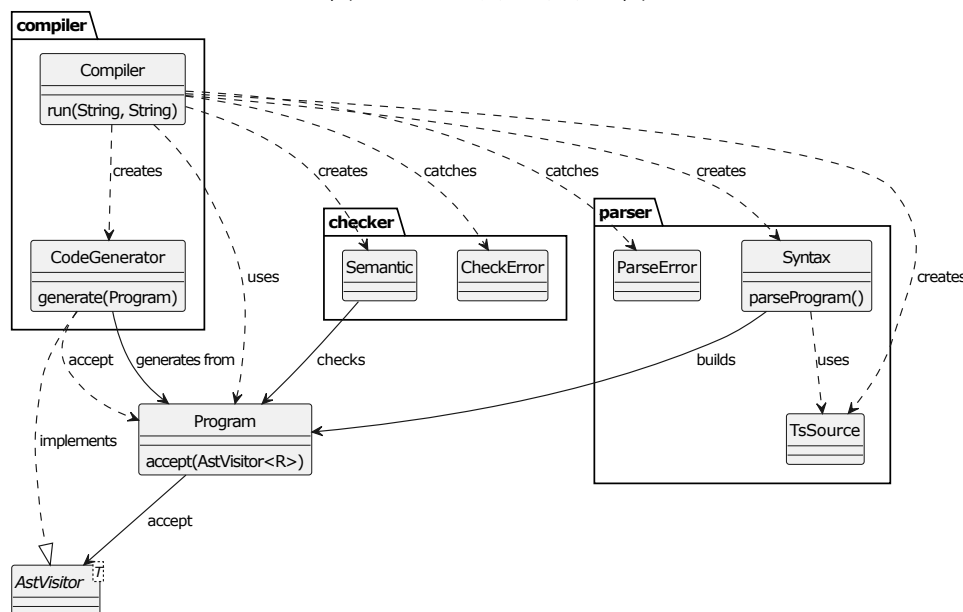
`Compiler.run(String, String)` は第一引数で指定した `ts` ファイルを読み込み、構文解析により `AST` を生成した後、意味解析で整合性を検査し、問題がなければ `CASL II` コードを生成して第二引数で指定した `cas` ファイルに出力する。出力時には生成コードの末尾に `data/cas/lib.cas` を連結し、実行可能な `CASL II` プログラムとする。入力に構文的または意味的誤りがある場合は、課題 2,3 と同形式のエラーメッセージを返して処理を終了する。入力ファイルが存在しない場合は `"File not found"` を返す。正常終了時は `"OK"` を返す。

2 課題達成の方針と設計

2.1 全体の処理の流れ

コンパイラのクラス図を図 1 に示す。本課題では、既存の字句・構文・意味解析器を利用し、`AST` を中間表現として `CASL II` コード生成までを一貫して実行する。また、コード生成では `CodeGenerator` が `AstVisitor` を実装し、`AST` を走査して命令列を生成する (`Visitor` パターン)。

図 1: コンパイラのクラス図

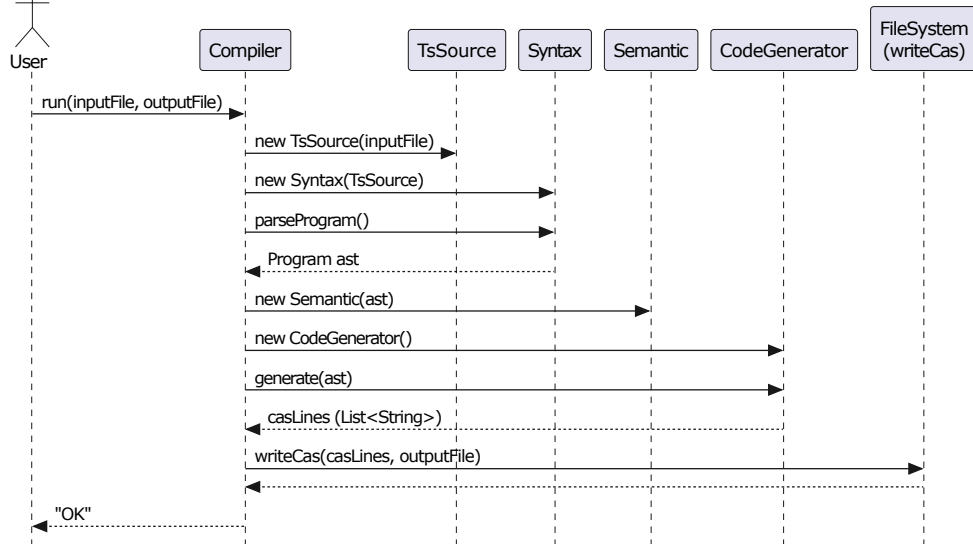


実行時の処理手順を図 2 に示す。図 2 に対応する処理の流れは次のとおりである。

1. `Compiler.run()` が呼び出され、入力 `ts` ファイルを対象としてコンパイルを開始する。
2. `TsSource` を生成して入力を読み込み、`Syntax` を生成する。
3. `Syntax.parseProgram()` を実行し、構文解析により `AST(Program)` を生成する。
4. `Semantic` を生成し、`AST` を入力として意味解析（宣言・スコープ・型整合性・手続き呼出の整合性の検査）を行う。
5. `CodeGenerator` を生成し、`generate(ast)` により `Visitor` として `Program.accept(this)` を起点に `AST` を走査し、`CASL II` 命令列を生成する。

6. 生成した命令列を `writeCas()` で `cas` ファイルへ書き出し、末尾に `lib.cas` を連結して実行可能な CASL II プログラムとする。

図 2: コンパイラのシーケンス図



2.2 出力するコードの構成と出力方法

本課題では、`CodeGenerator.generate(Program)` が CASL II コード全体を組み立てる。図 3 に示すように、生成コードは (1) ヘッダ、(2) メイン部、(3) 手続き定義部、(4) データ領域、(5) フッタの順に配置する。`generate` 内では、初期化後に `emitHeader()` を出力し、`ast.accept(this)` を起点として `Visitor` でメイン部の命令列を生成する。メイン部の末尾には `RET` を付加し、その後に `procOut` に蓄積した各手続きの命令列を連結する。最後に `emitData()` で変数領域・文字列定数領域・`LIBBUF` を蓄積し、`emitFooter()` により `END` を蓄積して完結させる。

また、コード生成とファイル出力は分離して実装している。`CodeGenerator` は生成結果を逐次ファイルへ書き込むのではなく、まず命令列を文字列のリストとしてメモリ上に蓄積する。具体的には、`generate` 内で `out` (メイン出力) および `procOut` (手続き出力) に対して `emit()` を通じて行単位で命令列を追加し、最終的にヘッダ、メイン、手続き、データ、フッタをこの順に連結した `List<String>` を返す。(図 3)

`Compiler.run()` は `generate` の戻り値として得られた `casLines` を `writeCas()` に渡し、ここで初めて `cas` ファイルとして書き出す。`writeCas()` では、生成した命令列に加えて `data/cas/lib.cas` を読み込み、その内容を末尾に追加して出力する。これにより、`MULT` や `DIV`、`WRTINT` などのライブラリルーチンを利用する前提の CASL II プログラムを、単体で実行可能な形で生成できる。

2.3 変数・配列・文字列定数の領域設計

本コンパイラでは、実行時に参照するデータをすべて CASL II のデータ領域としてプログラム末尾に配置する。`emitData()` により `VAR` (変数領域)、`CHAR0`, `CHAR1`, ... (文字列定数)、`LIBBUF` (入出力バッファ) をこの順に出力し、AST 走査中に収集した情報から、確保量 (`varCount`) やラベル (`strLabel`) を確定する。

CASL	START	BEGIN	;	(1) Header
BEGIN	LAD	GR6,0	;	
	LAD	GR7,LIBBUF	;	
	...			(2) Main
	RET		;	
PROC0	NOP		;	(3) Procedures
	...			
	RET		;	
...				
VAR	DS	<varCount>	;	(4) Data
CHAR0	DC	'...'	;	
...				
LIBBUF	DS	256	;	
	END		;	(5) Footer

図 3: CASL II プログラムの構成

2.3.1 変数領域 VAR の割当

変数は VAR DS <varCount> として連続領域を一括確保する方式とした。visitVarDecl で宣言を処理する際、識別子名に対して varOffset (名前 → 先頭オフセット) を登録し、確保語数を表す varCount を増加させることで領域割当を行う。通常の変数は 1 語を確保し、配列は要素数 len = upper-lower+1 を計算して len 語を連続確保する。このように宣言順にオフセットを付与することで、後続の参照・代入は「VAR + オフセット」の形に統一できる。

2.3.2 配列の下限と添字補正

配列は下限が 0 とは限らないため、添字をそのまま領域オフセットに用いることはできない。そこで配列名に対して下限値を arrayLower に保存し、配列先頭の割当位置 base (=varOffset) と下限 lower から k = base-lower を導入する。実行時に評価した添字 i に対して i+k を計算すれば、VAR 内で参照すべき実オフセットが得られる。この補正は visitArrayRef および visitArrayAccess で行い、連続確保した配列領域へ正しくアクセスできるようにした。

2.3.3 参照・代入の統一 (VAR 基準のオフセットアクセス)

参照・代入はすべて VAR を基準としたオフセットアクセスで統一した。通常変数の参照 (visitVarRef) では lookupOffset(name) で得たオフセットを用いて LD GR1, VAR, GR2 により値を読み出し、スタックへ PUSH する。代入 (visitVar) では右辺式を評価して POP した値を、同一オフセットへ ST で格納する。配列についても、添字補正後のオフセットを用いて LD/ST ..., VAR, GR2 を行う点は同じであり、データアクセス規約を単純化できる。

2.3.4 文字列定数の配置と共有 (CHARi)

文字列定数は重複定義を避けるために strLabel (文字列 → ラベル) で一元管理し、初出時に internStringLabel が CHAR0, CHAR1, ... を採番して割り当てる。emitData() では登録された順に CHARi DC '...' を出力し、参照側ではラベル名をアドレスとして利用する。また visitLiteral では、長さ 2 以上の文字列は「長さ」と「先頭アドレス」の 2 語をスタックに積むことで WRTSTR などのライブラリ呼出形式に合わせている。一方、長さ 1 の文字列は文字 (CHAR) として扱い、1 語の値として PUSH し、WRTCH に渡せる形に統一した。

2.3.5 入出力バッファ LIBBUF

入出力ライブラリが作業領域として利用するバッファとして LIBBUF DS 256 を確保する。ヘッダ部で LAD GR7, LIBBUF を行い、GR7 にバッファ先頭を設定することで、ライブラリルーチンが前提とする実行環境を整えている。

2.4 式評価と条件分岐

本コンパイラでは、式 (Expr) の評価結果をスタック上で受け渡す方式を採用した。各 visit メソッドは「自分が担当する式を評価したら、結果をスタックに PUSH する」ことを基本規約とし、複合式は子ノードを先に走査して値を積み、その後に POP して演算を行い、再び PUSH する。この規約により、ネストした式でも評価順序が明確になり、コード生成を局所的に実装できる。また、条件式の真偽値は #FFFF (真) と #0000 (偽) で表現し、if や while の分岐条件と整合するように統一した。

2.4.1 式評価の基本 (スタック規約)

二項演算では left, right の順に accept して値をスタックに積み、右辺、左辺の順に POP して計算する。例えば加算は ADDA, 減算は SUBA で行い、計算結果を PUSH して上位の式へ返す。単項演算も同様に、子式を評価して POP した値に対して演算を適用し、結果を PUSH する。符号反転 (NEG) は 0 - x を SUBA で実現し、論理否定 (NOT) は XOR ..., =#FFFF により #0000 と #FFFF を反転させる。

2.4.2 算術演算とライブラリ呼出

ADD/SUB は CASL II 命令のみで完結するが、MUL/DIV/MOD はライブラリルーチンに委譲する。二項演算のオペランドを GR1, GR2 に取り出した後、CALL MULT や CALL DIV を呼び出し、戻り値レジスタに格納された結果を PUSH して式の値とする。MOD については DIV 呼出後の剰余に対応するレジスタ (実装では GR1) を PUSH することで実現している。このように、式の値は常にスタック上にあるという前提を維持したまま、複雑な演算のみをライブラリに任せる構成とした。

2.4.3 比較演算と真偽値生成

比較演算 (EQ, NEQ, LT, LE, GT, GE) は、左右の値を POP した後に比較命令を実行し、分岐命令を用いて真偽値 (#FFFF/#0000) を生成してスタックに積む。例えば EQ では比較後に JZE で真分岐し、真の場合は #FFFF, 偽の場合は #0000 を LD して PUSH する。このとき、複数箇所で比較が発生してもラベルが衝突しないよう、jumpId により TRUE<i>, BOTH<i> 等のラベルを採番している。

また、本実装では比較対象が真偽値か整数かによって比較命令を切り替える。真偽値同士の比較は論理比較として CPL を用い、整数同士の比較は算術比較として CPA を用いる。この判定には、宣言処理で構築した varKind を参照し、左右の式の種別を exprKind で求めた上で isBoolCompare により分岐している。

2.4.4 条件分岐 (if)

if 文では、まず条件式を評価してスタックに積み、POP した値が #0000 かどうかで分岐する。具体的には CPA GR1, =#0000 の後、偽 (0) のとき JZE で ELSE<i> (または ENDIF<i>) へ分岐し、真 (非 0) のとき then 部を実行する。else 部が存在する場合は then 部の末尾で ENDIF<i> へ JUMP し、その間に ELSE<i> ラベルを配置して else 部を実行する。ラベルは jumpId により一意に採番し、入れ子構造でも正しく展開できる。

2.4.5 反復 (while)

while 文では LOOP<i> をループ先頭、ENDLP<i> を終了位置としてラベルを配置する。先頭で条件式を評価し、#0000 (偽) なら ENDLP<i> へ分岐してループを終了し、真なら本体を実行した後に LOOP<i> へ戻る。この構成により、構造化された反復構文を CASL II の条件分岐命令列へ直接対応付けている。

以上のように、式評価を「結果をスタックに積む」という規約で統一し、真偽値を #FFFF/#0000 に正規化することで、比較演算と if/while の分岐処理を一貫した枠組みで実装できた。図 4 は、if-else 文と while 文の分岐を実現している部分である。

... 条件式の計算 ...	LOOP<i> NOP
POP GR1	... 条件式の計算 ...
CPA GR1, =#0000	POP GR1
JZE ELSE<i>	CPA GR1, =#0000
... then 部 ...	JZE ENDLP<i>
JUMP ENDIF<i>	... body 部 ...
ELSE<i> NOP	JUMP LOOP<i>
... else 部 ...	ENDLP<i> NOP
ENDIF<i> NOP	

図 4: if-else および while 文の構成

2.5 手続きと引数の扱いとスコープ管理

本コンパイラでは、手続きを CASL II の CALL と RET により実現し、引数はスタックを介して受け渡す方式を採用した。ただし手続き本体では、引数を含む識別子参照をすべて VAR 基準のオフセットアクセスに統一したい。そこで、呼出直後に仮引数をスタックから取り出して VAR 領域へコピーし、スタック上に残る戻り番地の位置を調整してから本体を実行する構成とした。また、手続きごとに識別子表 (varOffset, varKind, arrayLower) を退避・復元することで、字句的スコープをコード生成段階でも維持する。

2.5.1 手続き定義の配置とラベル管理

手続き定義はメイン部の後方にまとめて配置する。visitProcedureDecl では出力先を currentOut により切り替え、手続きの命令列は procOut に蓄積する。generate の最後で out.addAll(procOut) によりメイン直後へ連結し、実行可能な構成 (メイン → 手続き定義 → データ領域) を保つ。手続き名とラベルは procLabel (名前 → PROC<i>) で管理し、CALL は対応ラベルへ分岐する。手続き本体の末尾には RET を付加して呼出元へ復帰する。

2.5.2 呼出側：引数の評価と CALL

呼出側 (visitProcCall) では、引数式を左から順に評価し、それぞれの値をスタックへ PUSH する。その後に CALL PROC<i> を発行する。この時点でスタック上には「引数列」と「CALL により積まれる戻り番地」が共存するため、手続き側ではこれらを整形してから本体を実行する必要がある。

2.5.3 仮引数処理に伴うスタック操作 (戻り番地・引数の後始末)

手続き呼出では、呼出側が引数をスタックに積んだ後に CALL を行うため、被呼出側の開始直後のスタックには「引数列」と「戻り番地」が混在する。本実装では、仮引数を手続き内でも通常変数と同じ規約 (VAR 基準のオフセット参照) で扱えるように、呼出直後に仮引数をスタックから VAR 領域へコピーする。コピー後は、引数領域をスタックから取り除き、戻り番地のみがスタック先頭に残る形へ整形することで、RET

が正しく復帰できる状態を保証する．図 5 左に処理の骨組み，右にスタック状態の遷移例を示す（図中は説明のため引数 2 個の場合を例示しているが，実装は `nParams` により一般の個数に対応する）．左図では `GR1` を `GR8`（`SP`）から初期化して仮引数列を指すように調整し（`ADDA GR1, =nParams`），`visitParam` が `LD/ST` を用いて各仮引数を `VAR[offset]` に格納する．最後に `LD GR1, 0, GR8` で戻り番地を退避し，`ADDA GR8, =nParams` で引数領域を破棄した後，`ST GR1, 0, GR8` により戻り番地を再配置することで，手続き本体がスタックを自由に利用した後でも `RET` により確実に呼出元へ復帰できる．

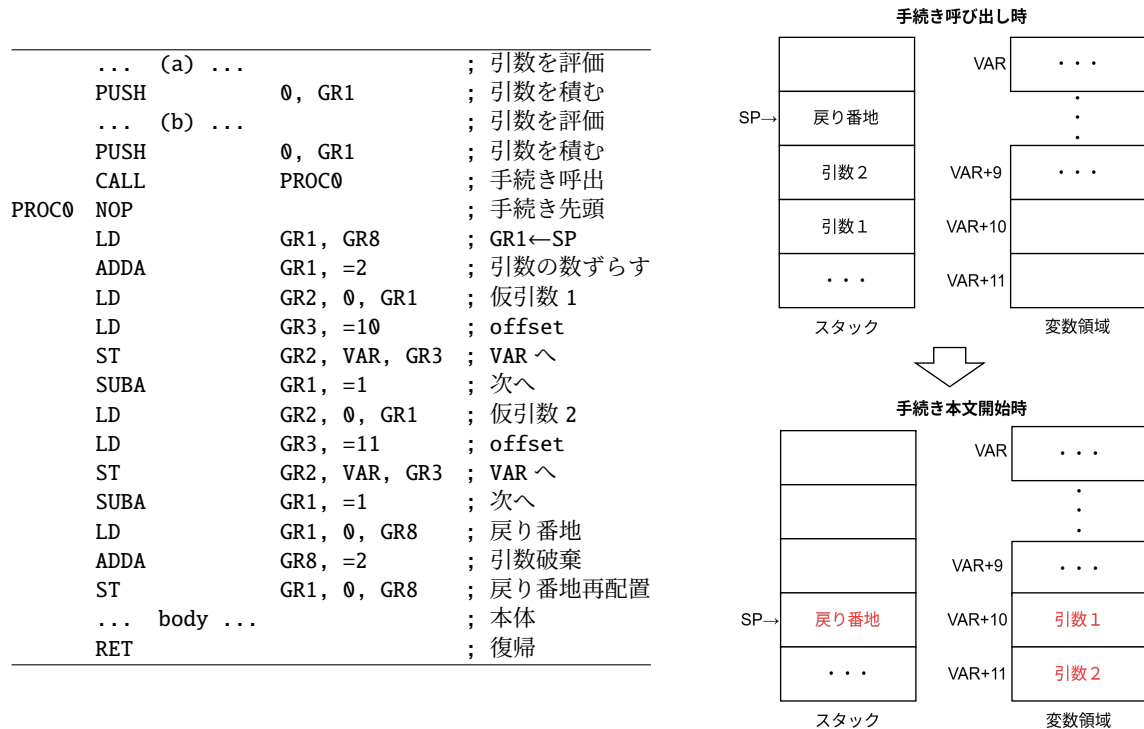


図 5: 仮引数処理に伴うスタック操作：コード（左）とスタック状態（右）

2.5.4 スコープの実現（識別子表の退避・復元）

手続きごとにローカル変数および仮引数を `VAR` 領域へ割り当てるため，識別子表（名前 → オフセット/型/配列下限）は手続き内部で一時的に拡張される．本実装では，`visitProcedureDecl` の先頭で外側スコープの情報（`varOffset`, `varKind`, `arrayLower`）を退避し，その内容をコピーした「手続き用の識別子表」を作ってから，手続き内部の宣言をこのコピー側に追加していく．具体的には，`visitVarDecl` や `visitParam` により，新たに現れた識別子をコピー側の `varOffset` 等へ登録し，手続き内だけで有効な名前解決（どの名前がどの領域を指すか）を成立させる．

手続き本体の生成が終わったら，退避しておいた元の識別子表を復元し，手続き用に作ったコピー側の識別子表は破棄（開放）する．この「コピーを拡張して使い，手続き終了時にコピーを捨てる」手順により，手続き内で追加したエントリ（ローカル変数・仮引数）は外側へ持ち出されず，スコープを抜けた識別子は以降のコード生成から参照できなくなる．その結果，手続き内で宣言した識別子が外側へ漏れないこと，および外側の同名識別子が手続き内で一時的に隠蔽されることを，自然に実現できる．

仮引数はスタック上に保持したまま参照するのではなく，手続き開始直後に `VAR` 領域へコピーし，以後は通常変数と同じ `VAR+offset` 規約で参照する．これにより，手続き内の式評価・代入・配列参照は常に「`VAR + offset`」に帰着し，スコープ管理は「どの名前がどの `offset` を持つか」を表す `varOffset` の管理に集中できるため，コード生成の見通しと保守性が向上する．

3 実装プログラム

3.1 コード生成器の全体構成

3.2 データ領域と変数アクセスの実装

3.3 制御構文と手続き呼出の実装

4 考察や工夫点

- 中間表現 (AST) を用いた設計の利点

本課題では、構文解析で生成した AST を中間表現として、意味解析とコード生成の両方で共通利用した。ノード種別 (Program, VarDecl, Binary, IfS など) に対して Visitor を適用することで、各処理をノードごとに局所化でき、生成規則の追加・修正が他部分へ波及しにくい構造になった。特に、構文 (木構造) と処理 (意味検査・コード生成) を分離できるため、機能追加やデバッグが容易になった。

- 評価規約の統一 (式評価・真偽値・制御構文の共通化)

式評価は「評価結果をスタックに PUSH して親へ返す」という規約に統一し、二項演算は子を先に評価して値を積み、POP して演算し、結果を再度 PUSH する方式とした。また真偽値を #FFFF (真) と #0000 (偽) に正規化し、if や while では「条件式を評価→#0000 なら分岐」の形に揃えた。この統一により、ネストした式・条件式・比較演算・制御構文を同一の枠組みで展開でき、コード生成の見通しと一貫性が向上した。さらに、手続き呼出でも仮引数を VAR へコピーして以後は通常変数と同規約で扱うことで、手続き本体は「スタック=式評価・一時値用」として自由に利用できる状態を保てるようにした。

- データ表現とメモリアクセス規約 (VAR 基準のオフセット設計)

変数・配列・仮引数をすべて VAR 領域に割り当て、「VAR + offset」で参照・代入する規約に統一した。visitVarDecl で varOffset を構築し、配列は arrayLower に下限を保持して添字補正 (base-lower+i) により連続確保領域へアクセスする。文字列定数は strLabel で共有し、CHAR<i> ラベルとして DC 領域へ集約することで重複定義を避けた。このように参照規約を単純化したことで、コード生成は名前解決と CASL 命令列生成を明確に分離できた。

- 今後の拡張性 (最適化・機能拡張の余地)

生成器が AST を基準に実装されているため、将来的に最適化や機能拡張を差し込みやすい。例えば覗き穴最適化として、局所的な命令列をパターンマッチして短縮できる。具体例として、PUSH 0, GR1 直後の POP GR1 のような無意味な往復の削除、LD GR1, =0 に続く ADDA GR1, =k を LD GR1, =k へ畳み込み、JUMP L; L NOP のような冗長ラベルの整理などが考えられる。また、将来的にレジスタ割当や定数畳み込み、分岐簡約などを導入する場合も、「AST 段階の最適化」または「CASL 出力後の最適化」を独立パスとして追加でき、現行実装を大きく崩さずに性能改善を進められる。

5 感想

本課題では、字句解析・構文解析・意味解析で得られた情報を用いて、最終的に CASL II の命令列を生成するまでを一貫して実装した。特に、機能を追加するたびにテストケースを 1 つずつ確認し、原因を切り分けながら修正していく作業は想像以上に手間がかかり、大変であった。一方で、小さなテストを積み重ねることで不具合の発生箇所を特定しやすくなり、最終的な安定動作につながることも実感できた。

また、構文解析で作成した AST を中間表現として用い、意味解析とコード生成を Visitor パターンで分離して実装したことで、処理の責務を整理しながら拡張できた点は有用だった。ノードごとの visit に局所的な規約（スタック規約や VAR+offset 参照規約）を持たせることで、生成規則を追跡しやすく、保守もしやすい構成になったと感じる。

さらに、生成された CASL II アセンブリコードを追いながら動作を確認する過程で、高級言語の構文が条件分岐やスタック操作、データ領域アクセスなどの具体的な命令列に落ちる様子を理解できた。コンパイラが出力したコードを手で読んで「プログラムが実際にどのように動いているか」を把握する経験は、普段のプログラミングでは得にくい学びであり、今後の低レイヤ理解やデバッグにも役立つと考える。